Cloud-containerization-and-DevOps

Lab Experiment 7: CI/CD using Jenkins, GitHub and Docker Hub

1. Aim

To design and implement a complete CI/CD pipeline using Jenkins, integrating source code from GitHub, and building & pushing Docker images to Docker Hub.

2. Introduction

CI/CD stands for Continuous Integration and Continuous Deployment. It automates the process of building, testing, and deploying applications. Tools like Jenkins, GitHub, and Docker Hub help in creating a complete pipeline for efficient software delivery.

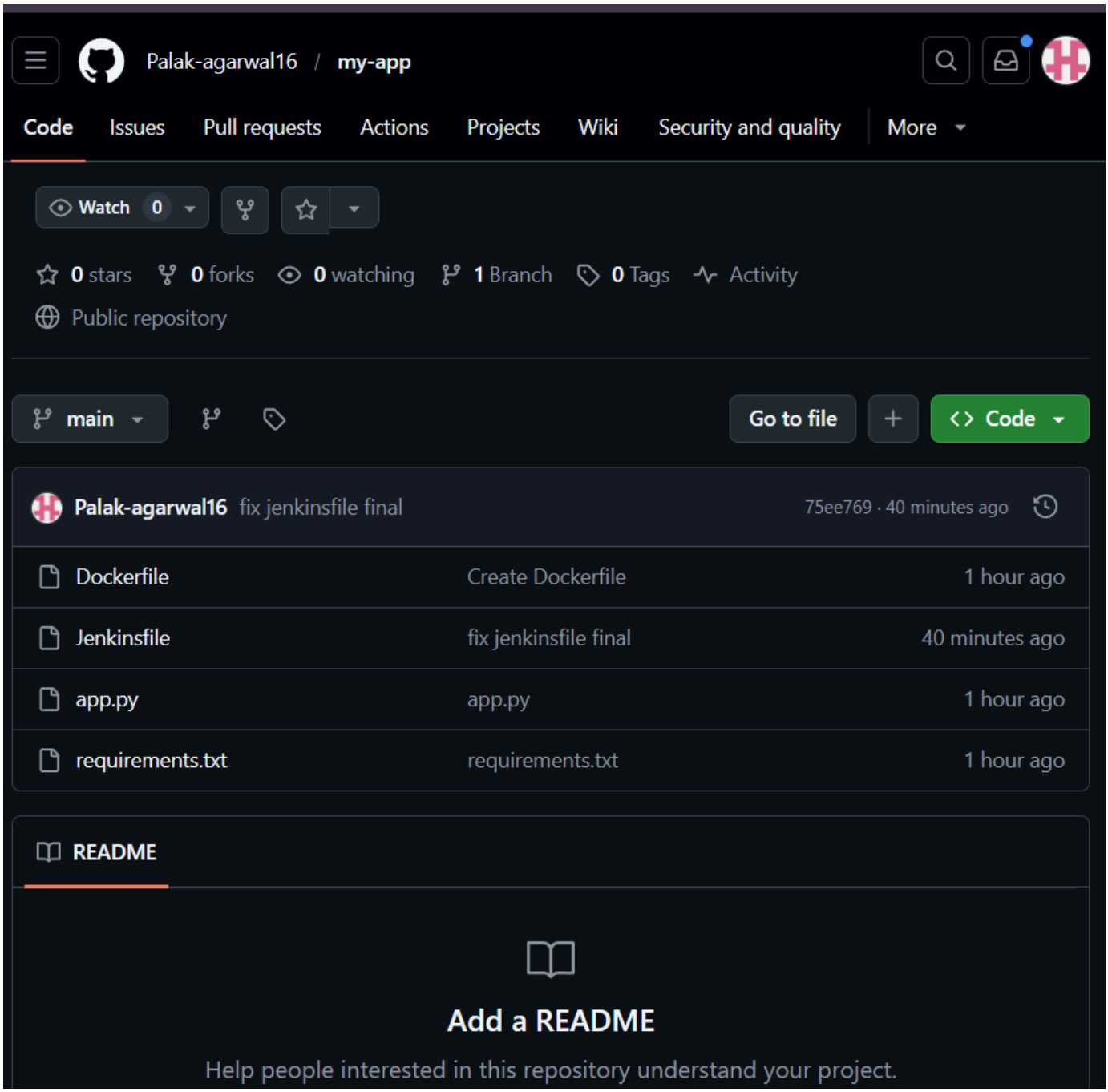
3. CI/CD Pipeline Architecture

Developer → GitHub → Jenkins → Docker Build → Docker Hub

4. Part A: GitHub Repository Setup (Source Code + Build Definition)

4.1 Create Repository

Create a repository on GitHub: my-app



4.2 Project Structure

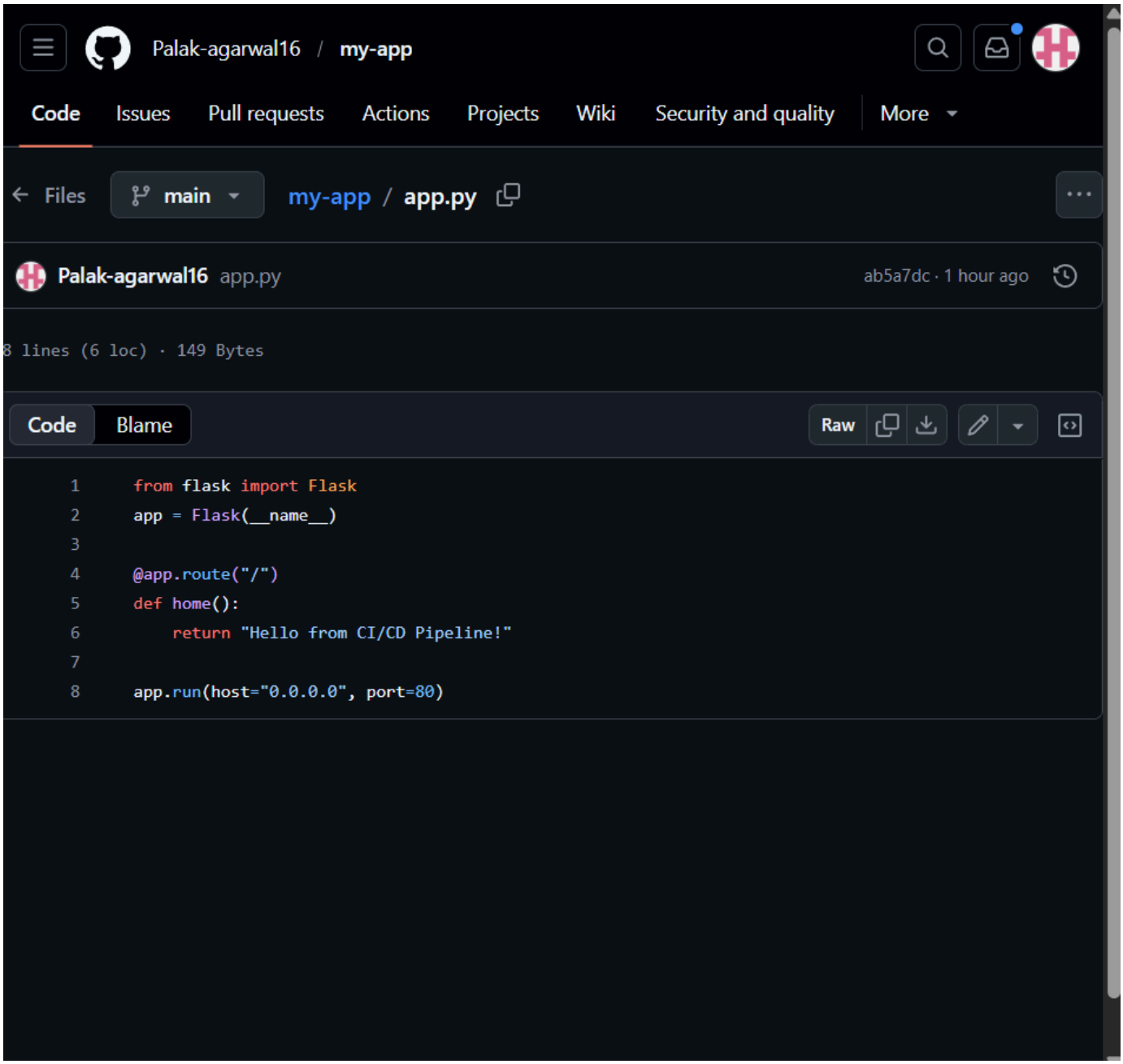
my-app/ ├── app.py ├── requirements.txt ├── Dockerfile ├── Jenkinsfile

4.3 Application Code

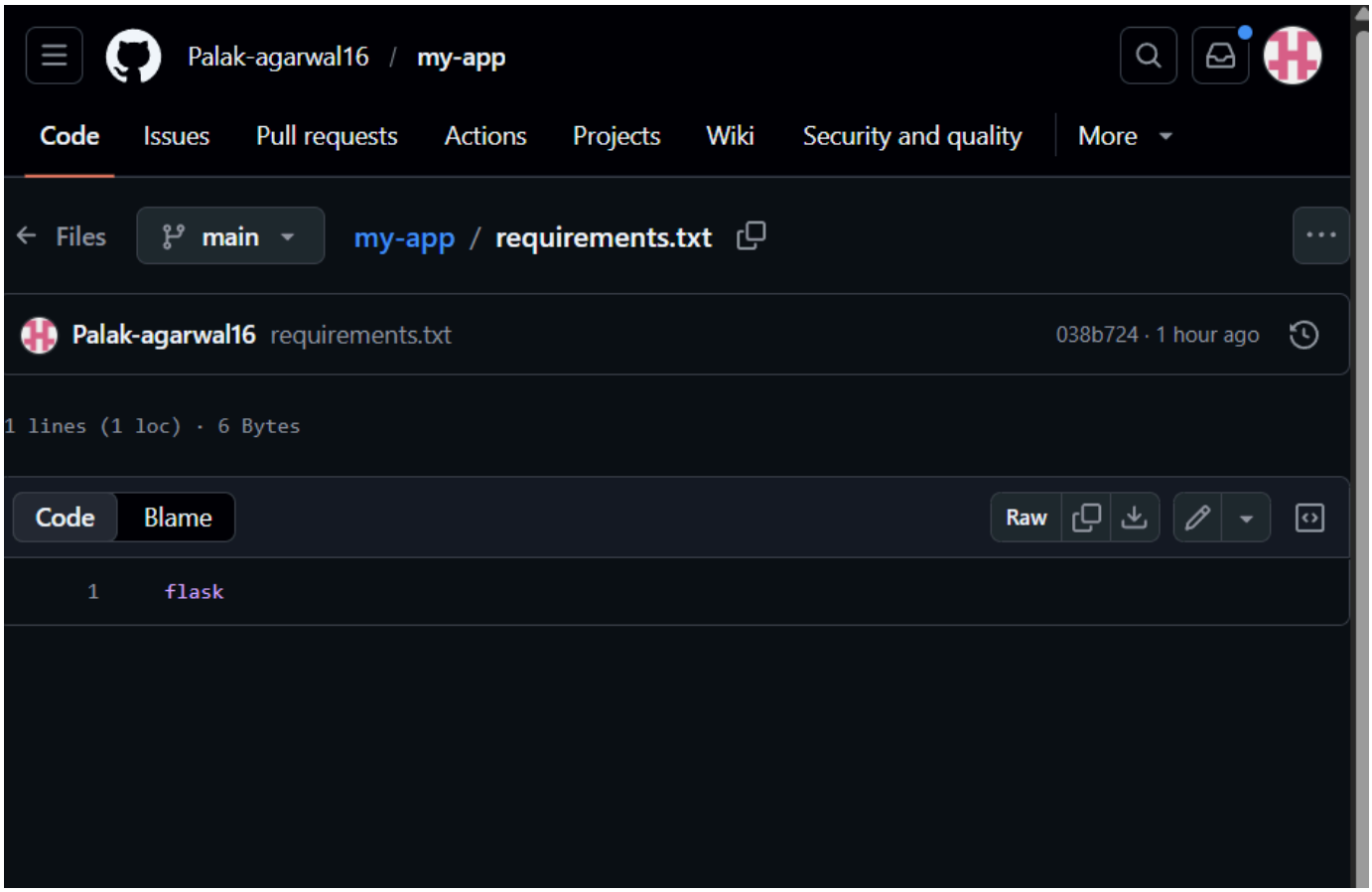
app.py from flask import Flask app = Flask(**name**)

@app.route("/") def home(): return "Hello from CI/CD Pipeline!" #return "Hello from CI/CD Pipeline!, my sapid is 123456"

app.run(host="0.0.0.0", port=80)



requirements.txt flask



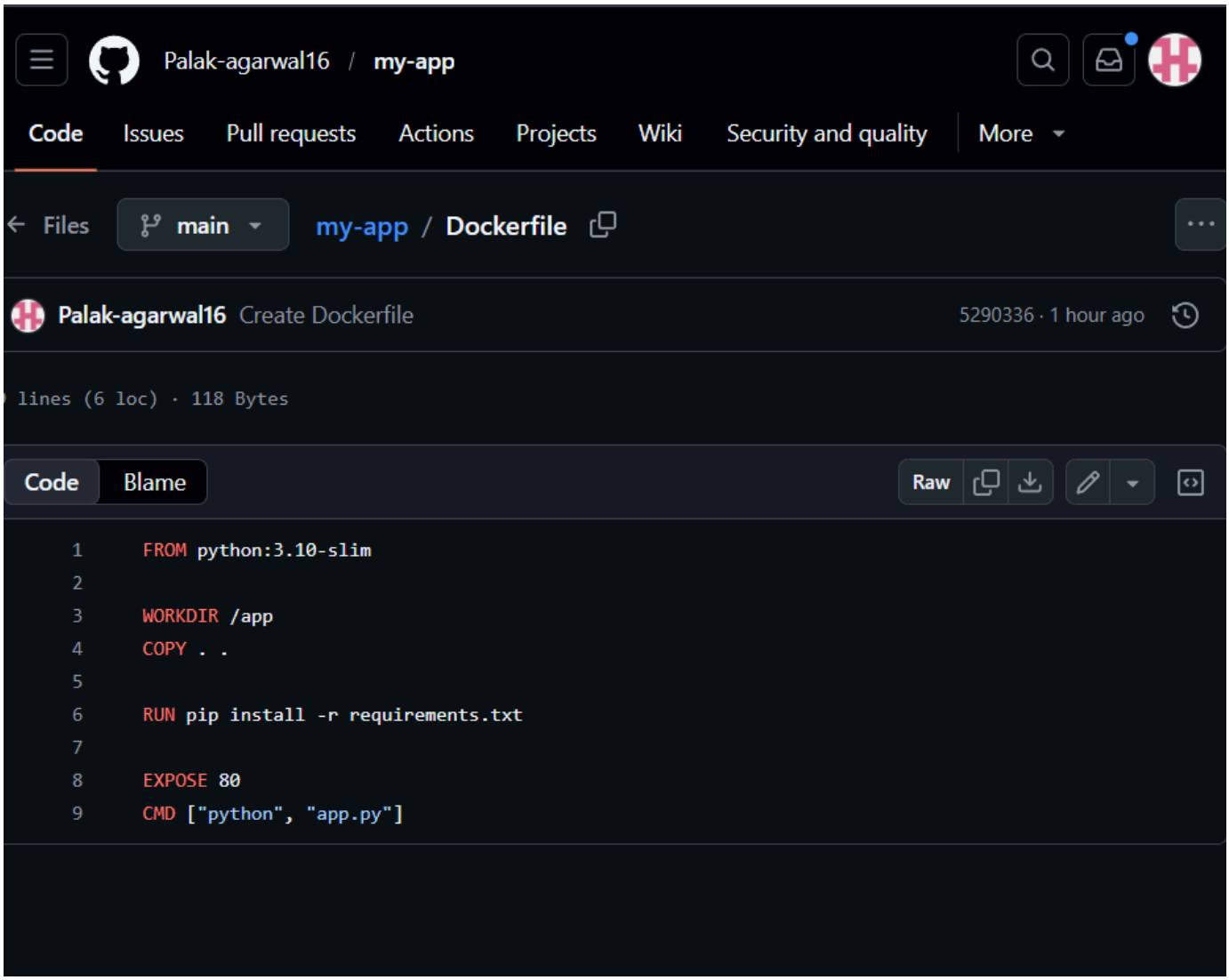
4.4 Dockerfile (Build Process)

FROM python:3.10-slim

WORKDIR /app COPY . .

RUN pip install -r requirements.txt

EXPOSE 80 CMD [“python”, “app.py”]



4.5 Jenkinsfile (Pipeline Definition in GitHub)

pipeline { agent any

```
environment {
    IMAGE_NAME = "your-dockerhub-username/myapp"
}

stages {

    stage('Clone Source') {
        steps {
            git 'https://github.com/your-username/my-app.git'
        }
    }

    stage('Build Docker Image') {
        steps {
            sh 'docker build -t $IMAGE_NAME:latest .'
        }
    }

    stage('Login to Docker Hub') {
```

```
            steps {
                withCredentials([string(credentialsId: 'dockerhub-token', variable: 'DOCKER_TC
                    sh 'echo $DOCKER_TOKEN | docker login -u your-dockerhub-username --passwor
                }
            }
        }

        stage('Push to Docker Hub') {
            steps {
                sh 'docker push $IMAGE_NAME:latest'
            }
        }
    } }
```

Files main my-app / Jenkinsfile

Palak-agarwal16 fix jenkinsfile final 75ee769 · 44 minutes ago

36 lines (30 loc) · 858 Bytes

Code Blame Raw

```
pipeline {
    agent any

    environment {
        IMAGE_NAME = "palakagarwal16/myapp"
    }

    stages {

        stage('Clone Source') {
            steps {
                git branch: 'main', url: 'https://github.com/Palak-agarwal16/my-app.git'
            }
        }

        stage('Build Docker Image') {
            steps {
                sh 'docker build -t $IMAGE_NAME:latest .'
            }
        }

        stage('Login to Docker Hub') {
            steps {
                withCredentials([string(credentialsId: 'dockerhub-token', variable: 'DOCKER_TOKEN')]
```

5. Part B: Jenkins Setup using Docker (Persistent Configuration)

5.1 Create Docker Compose File

version: '3.8'

services: jenkins: image: jenkins/jenkins:lts container_name: jenkins restart: always ports: - "8080:8080" - "50000:50000" volumes: - jenkins_home:/var/jenkins_home - /var/run/docker.sock:/var/run/docker.sock user: root

volumes: jenkins_home:

```
version: '3.8'

services:
  jenkins:
    image: jenkins/jenkins:lts
    container_name: jenkins
    restart: always
    ports:
      - "8080:8080"
      - "50000:50000"
    volumes:
      - jenkins_home:/var/jenkins_home
      - /var/run/docker.sock:/var/run/docker.sock
    user: root

volumes:
  jenkins_home:
```

5.2 Start Jenkins

docker-compose up -d

```
palakagarwal_16@LAPTOP-2SDBVCPO:/mnt/c/Users/hp/docker-lab$ docker-compose u
p -d
WARN[0000] /mnt/c/Users/hp/docker-lab/docker-compose.yml: the attribute `ver
sion` is obsolete, it will be ignored, please remove it to avoid potential c
onfusion
[+] up 16/16
 ✔Image jenkins/jenkins:lts      Pulled                                85.1s
 ✔Volume docker-lab_jenkins_home Created                               0.0s
 ✔Container jenkins              Started                               1.3s
```

Access: http://localhost:8080

5.3 Unlock Jenkins

docker exec -it jenkins cat /var/jenkins_home/secrets/initialAdminPassword

```
palakagarwal_16@LAPTOP-2SDBVCPO:/mnt/c/Users/hp/my-app$ docker exec -it jenk
ins cat /var/jenkins_home/secrets/initialAdminPassword
642decc4bb0a4feb99ce6da8f41d700f
```

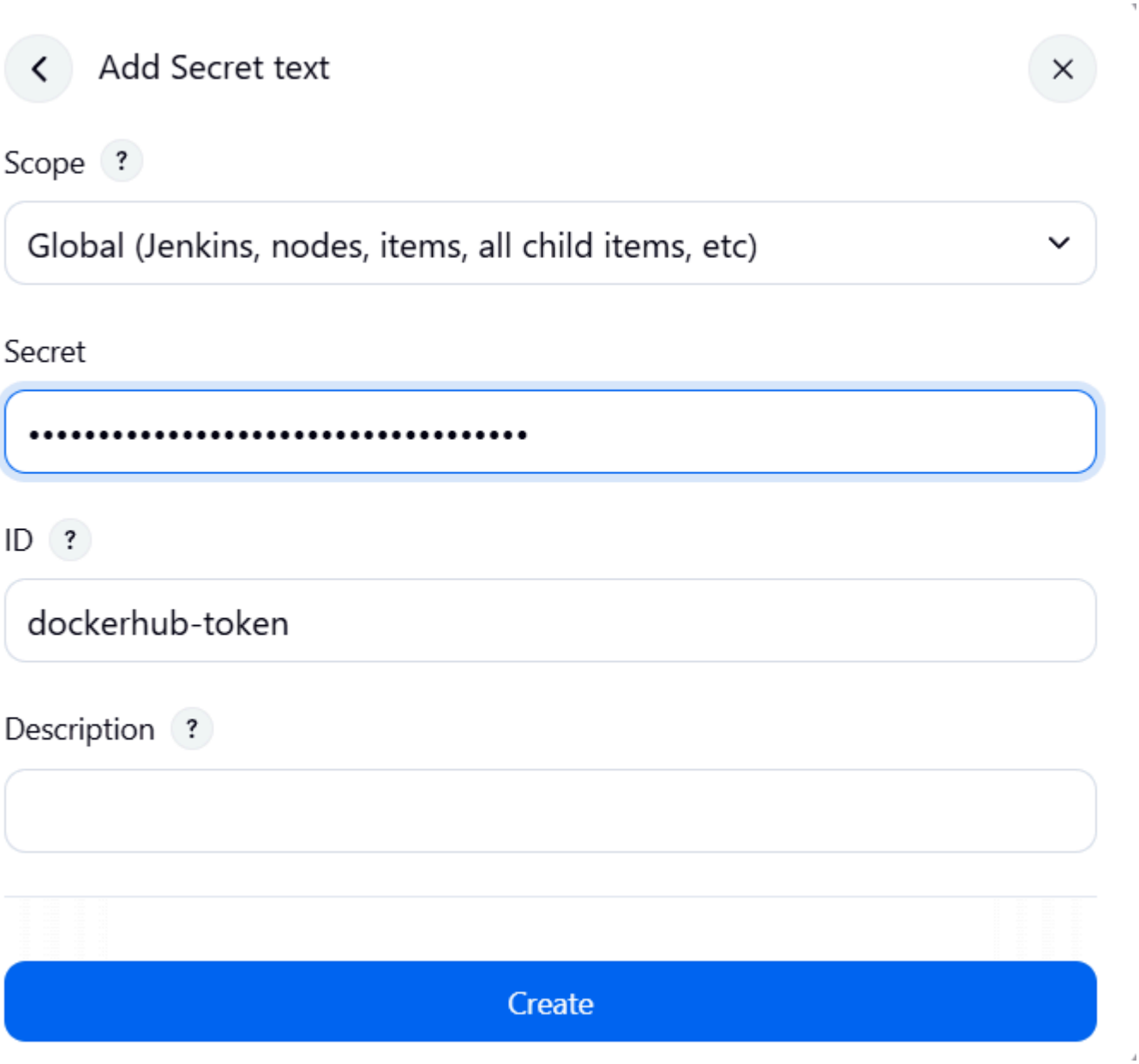
5.4 Initial Setup

- Install suggested plugins
- Create admin user

6. Part C: Jenkins Configuration

6.1 Add Docker Hub Credentials

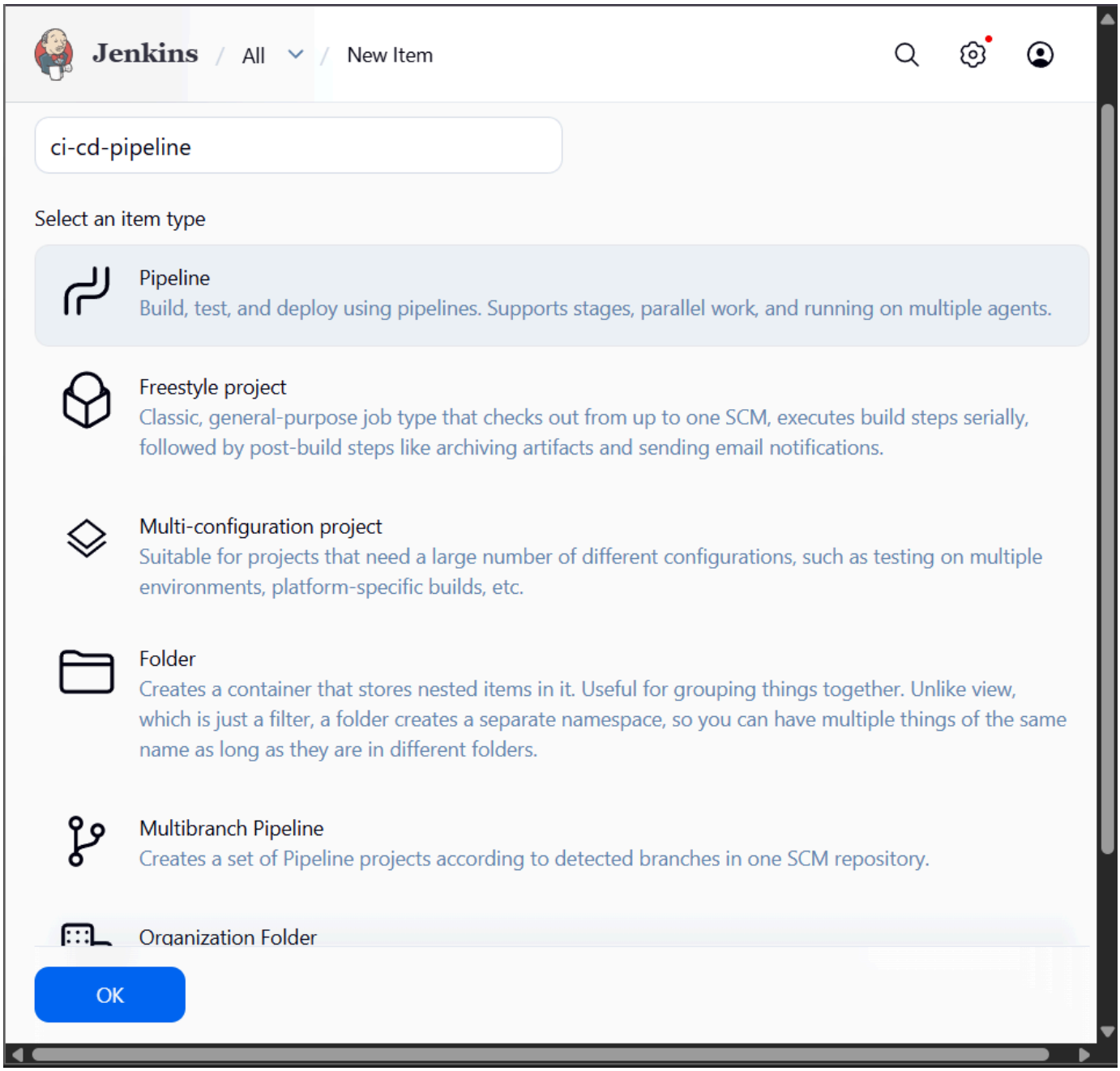
- Type: Secret Text
- ID: dockerhub-token
- Value: Docker Hub Access Token



6.2 Create Pipeline Job

Pipeline script from SCM

- SCM: Git
- Repo URL: your GitHub repo
- Script Path: Jenkinsfile

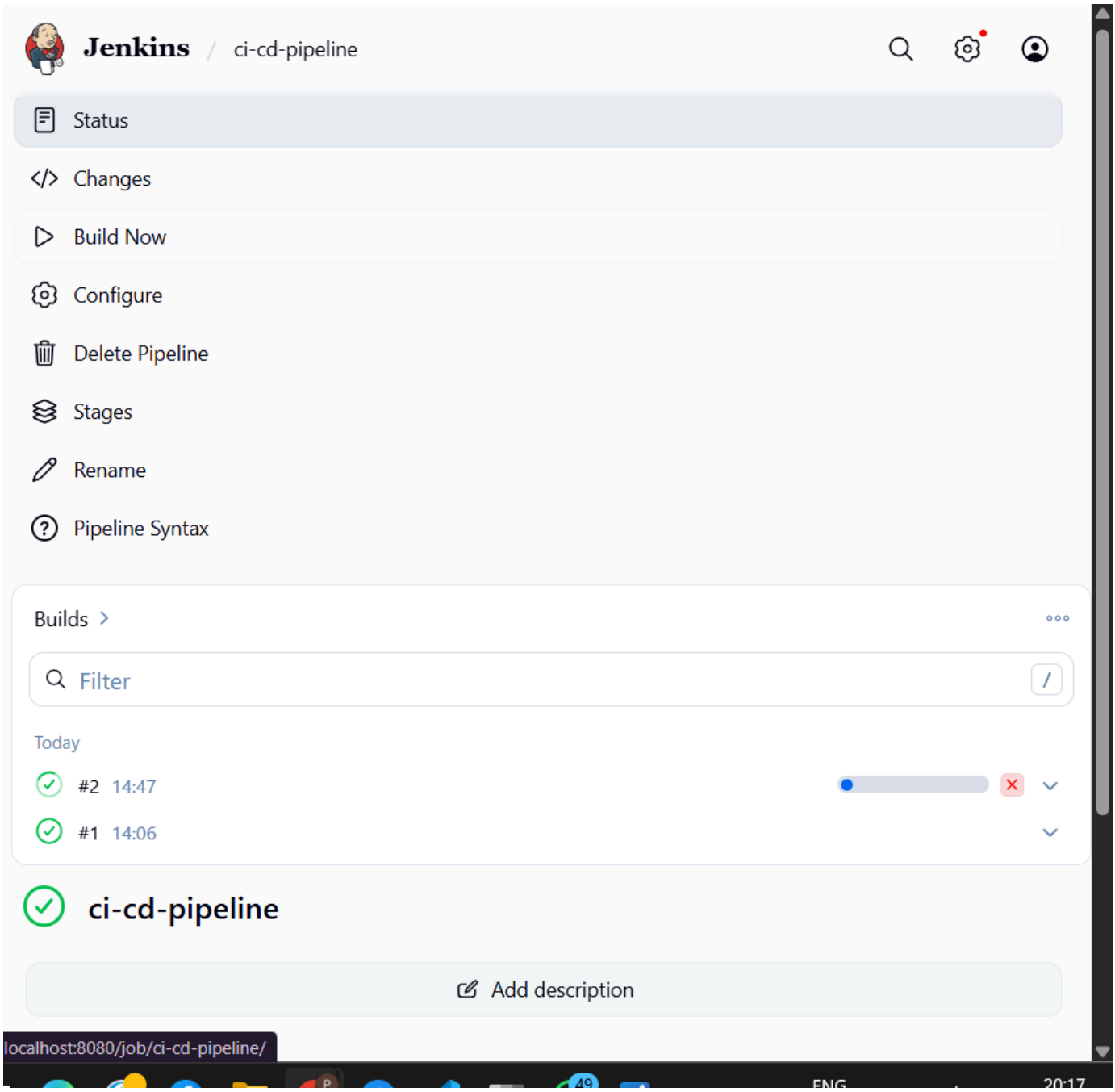


7. Part D: Trigger Mechanism (Manual Execution)

In this experiment, the pipeline was triggered manually from Jenkins dashboard.

Steps:

1. Open Jenkins Dashboard
2. Select pipeline job
3. Click Build Now
4. Pipeline executes

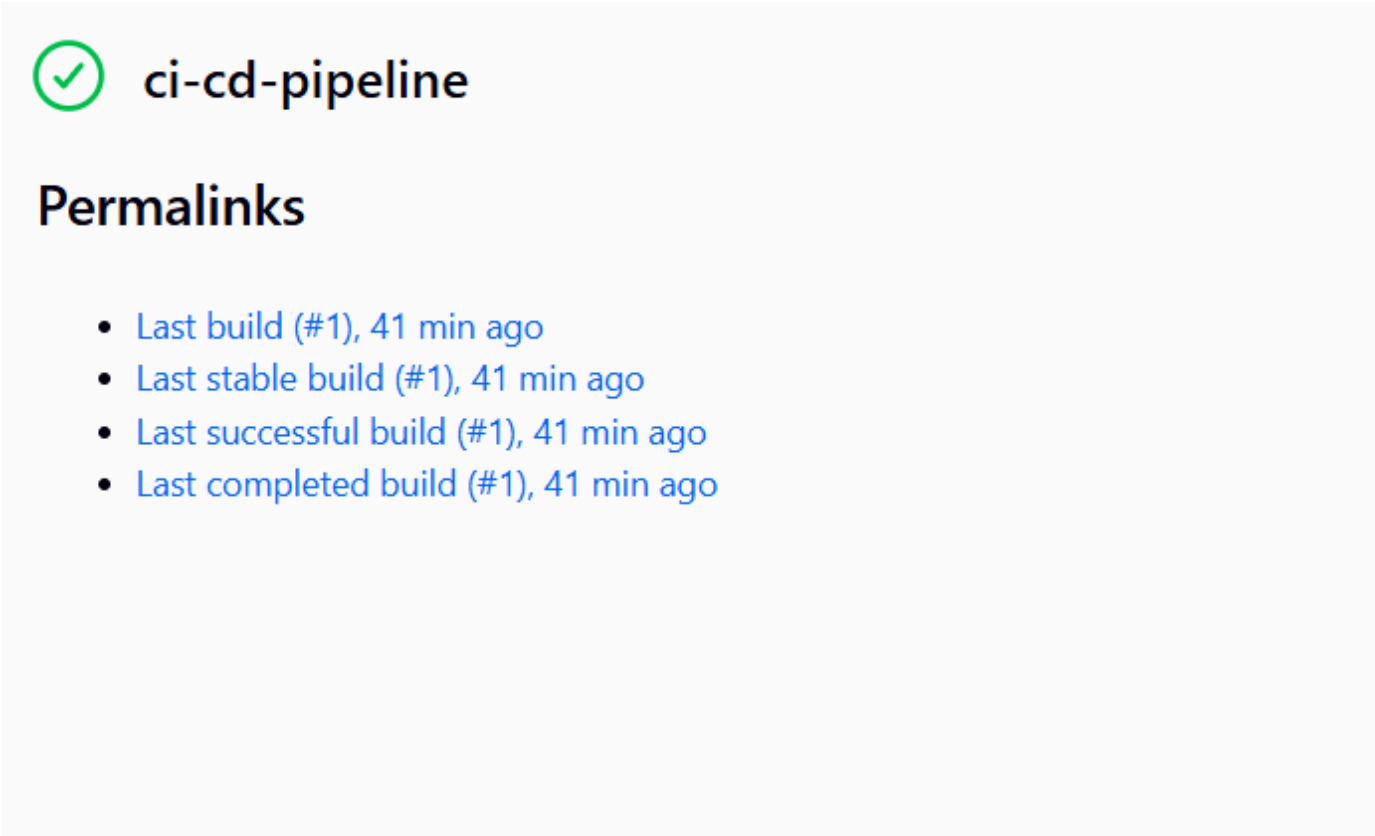


8. Part E: Execution Flow (Stage-wise)

Stage 1: Code Push Developer updates code in GitHub

```
palakagarwal_16@LAPTOP-2SDBVCPO:/mnt/c/Users/hp/my-app$ git add .
palakagarwal_16@LAPTOP-2SDBVCPO:/mnt/c/Users/hp/my-app$ git commit -m "fix j
enkinsfile final"
[main 75ee769] fix jenkinsfile final
 1 file changed, 1 insertion(+), 1 deletion(-)
palakagarwal_16@LAPTOP-2SDBVCPO:/mnt/c/Users/hp/my-app$ git push
Username for 'https://github.com': Palak-agarwal16
Password for 'https://Palak-agarwal16@github.com':
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 12 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 322 bytes | 16.00 KiB/s, done.
Total 3 (delta 2), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/Palak-agarwal16/my-app.git
   17c6ddf..75ee769  main -> main
```

Stage 2: Triggering GitHub sends event to Jenkins



Stage 3: Jenkins Pipeline Execution

Clone Stage: Pulls latest code from GitHub

```
Started by user Palak Agarwal
Obtained Jenkinsfile from git https://github.com/Palak-agarwal16/my-app.git
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /var/jenkins_home/workspace/ci-cd-pipeline
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Declarative: Checkout SCM)
[Pipeline] checkout
Selected Git installation does not exist. Using Default
The recommended git tool is: NONE
No credentials specified
 > git rev-parse --resolve-git-dir /var/jenkins_home/workspace/ci-cd-pipeline/.git # timeout=10
Fetching changes from the remote Git repository
 > git config remote.origin.url https://github.com/Palak-agarwal16/my-app.git # timeout=10
Fetching upstream changes from https://github.com/Palak-agarwal16/my-app.git
```

Build Stage: Docker builds image

```
palakagarwal_16@LAPTOP-2SDBVCPO:/mnt/c/Users/hp/my-app$ docker build -t pala
kagarwal16/myapp:latest .
[+] Building 24.6s (8/9)                                        docker:default
 => [internal] load build definition from Dockerfile                      0.3s
 => => transferring dockerfile: 155B                                      0.1s
 => [internal] load metadata for docker.io/library/python:3.10-slim       2.8s
 => [auth] library/python:pull token for registry-1.docker.io             0.0s
 => [internal] load .dockerignore                                         0.1s
 => => transferring context: 2B                                           0.0s
 => [1/4] FROM docker.io/library/python:3.10-slim@sha256:26fc00cf5fd6     0.1s
 => => resolve docker.io/library/python:3.10-slim@sha256:26fc00cf5fd6     0.1s
 => [internal] load build context                                         0.7s
 => => transferring context: 40.15kB                                      0.7s
 => CACHED [2/4] WORKDIR /app                                             0.0s
 => [3/4] COPY . .                                                        0.2s
 => [4/4] RUN pip install -r requirements.txt                            20.3s
 => => # Collecting click>=8.1.3
 => => #   Downloading click-8.3.2-py3-none-any.whl (108 kB)
 => => #      ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 108.4/108.4 kB 132.0 kB
 => => # /s eta 0:00:00
 => => # Collecting itsdangerous>=2.2.0
 => => #   Downloading itsdangerous-2.2.0-py3-none-any.whl (16 kB)
```

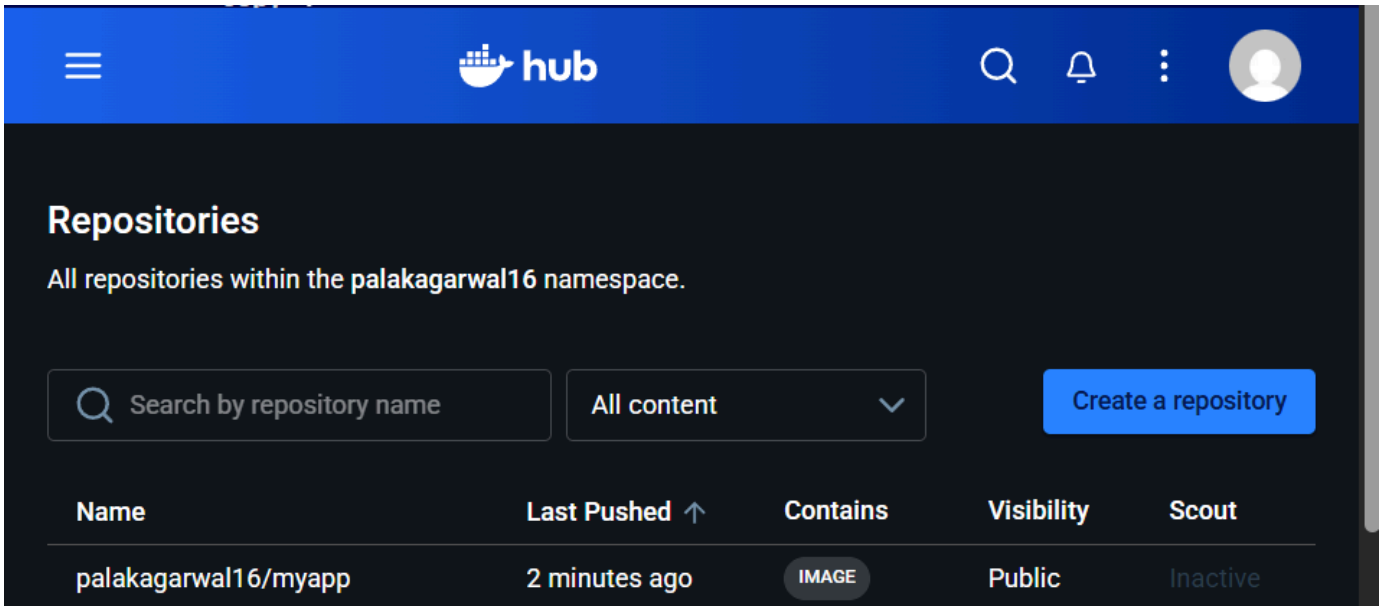
Auth Stage: Jenkins logs into Docker Hub

```
sha256:80e951644ad6fdf4ec08d77b06d3d3314f3c1692d1bb12a5465606218651d23b 0.1s done
#10 exporting manifest list
sha256:a5f5a44524466caedab1b18c7d9cc6b5450ff3f44c4b1d8f287e8179011705c7 0.0s done
#10 naming to docker.io/palakagarwal16/myapp:latest 0.0s done
#10 unpacking to docker.io/palakagarwal16/myapp:latest
#10 unpacking to docker.io/palakagarwal16/myapp:latest 2.2s done
#10 DONE 4.7s
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Login to Docker Hub)
[Pipeline] withCredentials
Masking supported pattern matches of $DOCKER_TOKEN
[Pipeline] {
[Pipeline] sh
+ echo ****
+ docker login -u palakagarwal16 --password-stdin
WARNING! Your password will be stored unencrypted in /root/.docker/config.json.
Configure a credential helper to remove this warning. See
https://docs.docker.com/engine/reference/commandline/login/#credentials-store

Login Succeeded
```

Push Stage: Image pushed to Docker Hub

```
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Push to Docker Hub)
[Pipeline] sh
+ docker push palakagarwal16/myapp:latest
The push refers to repository [docker.io/palakagarwal16/myapp]
ddf3b231f567: Waiting
bdd96403af7b: Waiting
6f466d9741b7: Waiting
bed04db721b4: Waiting
5435b2dcdf5c: Waiting
8a40b8b01719: Waiting
```

Stage 4: Artifact Ready Docker image available globally



/var/run/docker.sock

```
palakagarwal_16@LAPTOP-2SDBVCPO:/mnt/c/Users/hp/my-app$ cat docker-compose.y
ml
version: '3.8'

services:
  jenkins:
    image: jenkins/jenkins:lts
    container_name: jenkins
    restart: always
    ports:
      - "8080:8080"
      - "50000:50000"
    volumes:
      - jenkins_home:/var/jenkins_home
      - /var/run/docker.sock:/var/run/docker.sock
    user: root

volumes:
  jenkins_home:
palakagarwal_16@LAPTOP-2SDBVCPO:/mnt/c/Users/hp/my-app$
```

9. Basic Pipeline Structure

pipeline { agent any

```
  stages {
      stage('Build') {
          steps {
              sh 'echo Hello'
          }
      }
  } }
```

Jenkins / ci-cd-pipeline / #2 / Stages

Search

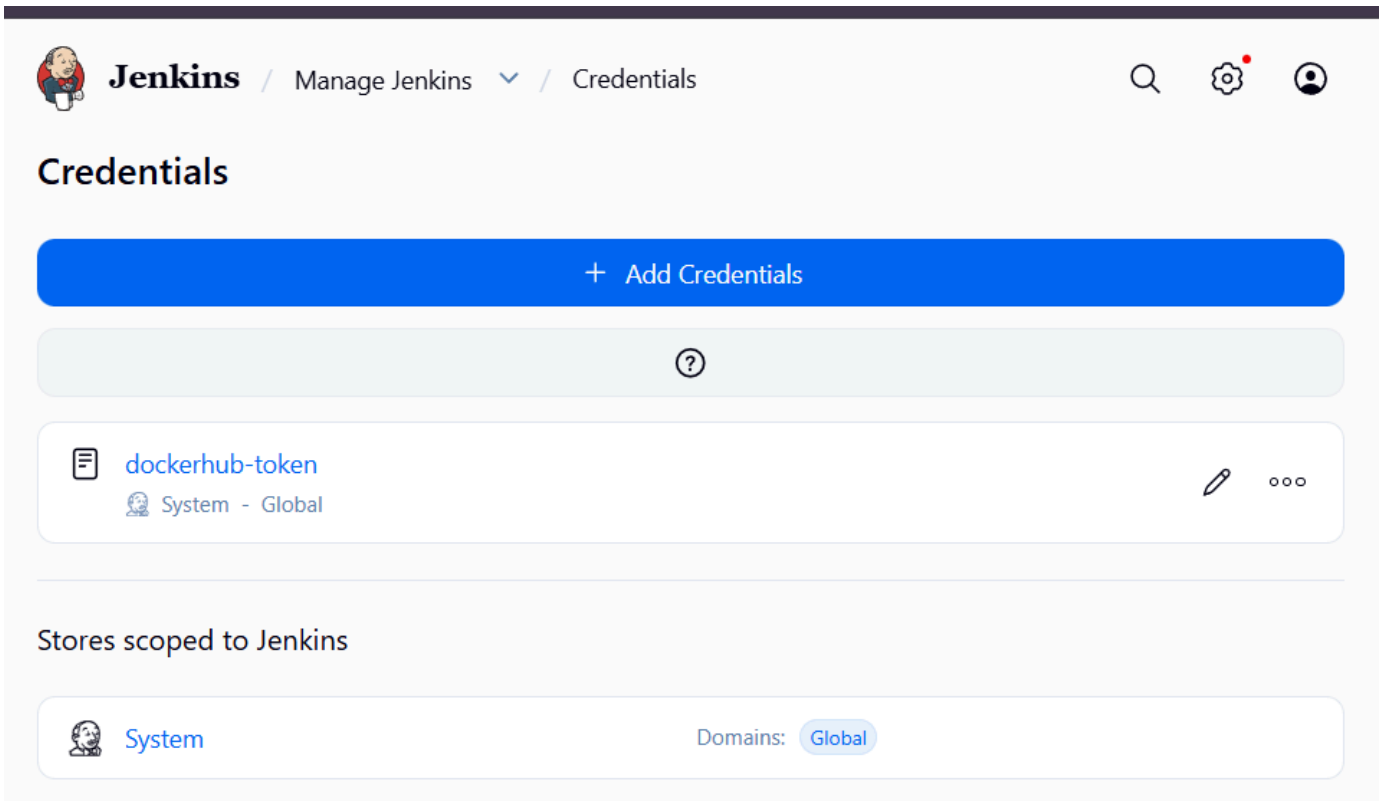
- Checkout SCM 1s
- Clone Source 1s
- Build Docker Image 33s
- Login to Docker Hub 3s
- Push to Docker Hub 9s

docker push $IMAGE_NAME:latest 9s

```
13 bed04db721b4: Waiting
14 5435b2dcdf5c: Waiting
15 8a40b8b01719: Waiting
16 935d60c8480a: Waiting
17 9833b846f421: Waiting
18 935d60c8480a: Waiting
19 9833b846f421: Waiting
20 ddf3b231f567: Waiting
21 bdd96403af7b: Waiting
22 6f466d9741b7: Waiting
23 bed04db721b4: Waiting
24 5435b2dcdf5c: Waiting
25 8a40b8b01719: Waiting
26 5435b2dcdf5c: Waiting
27 8a40b8b01719: Waiting
28 935d60c8480a: Waiting
29 9833b846f421: Waiting
30 ddf3b231f567: Waiting
```

Code Blame Raw

```
    environment {
        IMAGE_NAME = "palakagarwal16/myapp"
    }

    stages {

        stage('Clone Source') {
            steps {
                git branch: 'main', url: 'https://github.com/Palak-agarwal16/my-app.git'
            }
        }

        stage('Build Docker Image') {
            steps {
                sh 'docker build -t $IMAGE_NAME:latest .'
            }
        }

        stage('Login to Docker Hub') {
            steps {
                withCredentials([string(credentialsId: 'dockerhub-token', variable: 'DOCKER_TOKEN')]
                    sh 'echo $DOCKER_TOKEN | docker login -u palakagarwal16 --password-stdin'
                }
            }
        }

        stage('Push to Docker Hub') {
            steps {
                sh 'docker push $IMAGE_NAME:latest'
            }
        }
```



10. Key Takeaways

- pipeline → stages → stage → steps defines structure
- sh executes shell commands
- git fetches code
- withCredentials secures secrets
- secrets are protected and temporary

11. Result

CI/CD pipeline executed successfully. Docker image built and pushed to Docker Hub.

12. Conclusion

Successfully implemented CI/CD pipeline using Jenkins, GitHub, and Docker Hub. Automation improved efficiency and reduced manual effort.